

**TECHNICAL
RESEARCH
REPORT**

*Institute for
Systems
Research*

**Hilbert R-Tree:
An Improved R-Tree Using Fractals**

by I. Kamel and C. Faloutsos

*The Institute for Systems
Research is supported by the
National Science Foundation
Engineering Research Center
Program (NSFD CD 8803012),
Industry and the University*

TR 93-19

Hilbert R-tree: An improved R-tree using fractals

*Ibrahim Kamel and Christos Faloutsos**

Department of CS
University of Maryland
College Park, MD 20742

February 18, 1993

Abstract

We propose a new R-tree structure that outperforms all the older ones. The heart of the idea is to impose a linear ordering on the data rectangles. This ordering has to be 'good', in the sense that it should cluster 'similar' data rectangles together, to minimize the area and perimeter of the resulting minimum bounding rectangles (MBRs). Among the orderings we tried, the '2D-c' method, the one that uses the (2d) hilbert value of the center of the rectangles, gives the best results.

For a static database, the proposed ordering achieves superior packing, outperforming older packing methods [25], and the best dynamic method (R^* -trees [3]). The savings are as high as 36% on real data.

More importantly, we introduce a dynamic variation, the *Hilbert R-tree*: Given the ordering, every node has a well-defined set of sibling nodes; thus, we can deploy the deferred splitting algorithms of the B^* -trees. By adjusting the split policy, the Hilbert R-tree can achieve as high utilization as desired. We show that a '3-to-4' split policy achieves good results, consistently outperforming the R^* -trees, with up to 28% savings on real data.

1 Introduction

One of the requirements for the database management systems (DBMSs) of the near future is the ability to handle spatial data. Spatial data arise in many applications, including: Cartography [28]; Computer-Aided Design (CAD) [23] [13]; computer vision and robotics [2]; traditional databases.

*Currently on sabbatical at IBM Almaden Research Center. This research was partially funded by the Systems Research Center (SRC) at the University of Maryland, and by the National Science Foundation under Grant IRI-8958546 (PYI), with matching funds from EMPRESS Software Inc. and Thinking Machines Inc.

where a record with k attributes corresponds to a point in a k -d space; temporal databases, where time can be considered as one more dimension [18]; scientific databases, with spatial-temporal data, etc.

In the above applications, one of the most typical queries is the *range query*: Given a rectangle, retrieve all the elements that intersect it. A special case of the range query is the *point query* or *stabbing query*, where the query rectangle degenerates to a point.

We focus on R-trees [14], which is one of the most efficient methods that support range queries. The most successful variant of R-trees seems to be the R^* -tree [3]. One of the main contributions there is the idea to defer the splitting, by choosing some rectangles from the overflowing node, and "force-reinserting" them.

The main idea in the present paper is to propose an ordering of the data rectangles. The consequences are important: using this ordering, each R-tree node has a well defined set of siblings; thus, we can use the B^* -tree algorithms for deferred splitting. By adjusting the split policy (2-to-3 or 3-to-4 etc) we can drive the utilization as close to 100% as desirable. Notice that the R^* -tree does not have control over the utilization, typically achieving an average of $\approx 70\%$.

The only requirement for the ordering is that it has to be 'good', that is, it should lead to small R-tree nodes. We experiment with several orderings, using fractals and specifically the Hilbert curve, and chose the best.

We also examine a static environment, where we pack the data rectangles into R-tree nodes according to our best ordering. In this case, the performance of our method is even better, since the utilization is 100%, maximizing the fanout and minimizing the height of the tree. For distinction, we refer to the static version by *Static Hilbert R-tree*. The dynamic version is called *Dynamic Hilbert R-tree*, or simply *Hilbert R-tree*.

The paper is organized as follow. Section 2 gives a brief description of the R-tree and its variants. In section 3 we introduce the Static Hilbert R-tree, along with alternative ordering schemes. Section 4 describes the Dynamic Hilbert R-tree. Section 5 presents our experimental results that compare the Hilbert R-tree with other R-tree variants. Section 6 gives the conclusions and directions for future research.

2 Survey

Several spatial access methods have been proposed. A recent survey can be found in [26]. These methods fall in the following broad classes: methods that transform rectangles into points in a higher dimensionality space [15]; methods that use linear quadrees [9] [1] or, equivalently, the z -ordering [22] or other space filling curves [7] [17]; and finally, methods based on trees (R-tree [14],

k-d-trees [4], k-d-B-trees [24], hB-trees [20], cell-trees [12] e.t.c.)

One of the most promising approaches in the last class is the R-tree [14]. It is the extension of the B-tree for multidimensional objects. A geometric object is represented by its minimum bounding rectangle (MBR). Non-leaf nodes contain entries of the form (R, ptr) where ptr is a pointer to a child node in the R-tree; R is the MBR that covers all rectangles in the child node. Leaf nodes contain entries of the form $(obj-id, R)$ where $obj-id$ is a pointer to the object description, and R is the MBR of the object. The main innovation in the R-tree is that father nodes are allowed to overlap. This way, the R-tree can guarantee at least 50% space utilization and remain balanced.

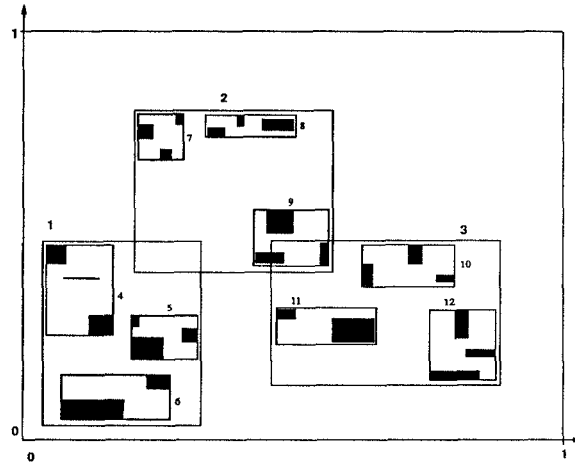


Figure 1: Data (dark rectangles) organized in an R-tree with fanout=3

Figure 1 illustrates data rectangles (in black), organized in an R-tree with fanout 3. Figure 2 shows the file structure for the same R-tree, where nodes correspond to disk pages. In the rest of

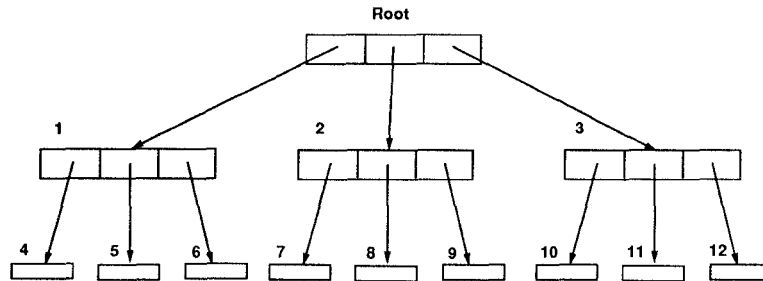


Figure 2: The file structure for the R-tree of the previous figure (fanout=3)

this paper, the term ‘node’ and the term ‘page’ will be used interchangeably. Guttman proposed three splitting algorithms, the *linear split*, the *quadratic split* and the *exponential split*. Their names come from their complexity; among the three, the quadratic split algorithm is the one that achieves

the best trade-off between splitting time and search performance.

Subsequent work on R-trees includes the work by Greene [10], the R^+ -tree [27], R-trees using Minimum Bounding Polygons [16], and finally, the R^* -tree [3], which seems to have the best performance among the R-tree variants. The main idea in the R^* -tree is the concept of *forced re-insert*, which is analog to the deferred-splitting in B-trees. When a node overflows, some of its children are carefully chosen and they are deleted and re-inserted, usually resulting in a R-tree with better structure.

All the above R-tree variants support insertions and deletions, and are thus suitable for *dynamic* environments. For a static environment, Roussopoulos and Leifker [25] proposed a method to build a packed R-tree that achieves (almost) 100% space utilization by sorting the data on the x or y coordinate of one of the corners of the rectangles. The sorted list of rectangles is scanned; successive rectangles are assigned to the same R-tree leaf node, until this node is full; then, a new leaf node is created and the scanning of the sorted list continues. However, its response time is worse than both R^* -tree and quadratic split R-tree for region queries and/or rectangular data. For the rest of this paper, we shall refer to this method by *lowx packed R-tree*. In our implementation of their method, we sort the rectangles according to their x value of the lower left corner (*lowx*). Sorting on any of the other three values gives similar results; thus our implementation does not impose an unfair disadvantage to the *lowx* packed R-tree.

3 Static Version - ordering of rectangles

In this section, we assume that the data are static, or that the frequency of modification is low. Our goal is to design a simple heuristic to construct an R-tree with 100% space utilization, which, on the same time, will have as good response time as possible. The benefits from such an effort are twofold: In addition to serving as a stepping stone for the dynamic version, a good packing method is useful per se:

Static data appear in several applications. For example, in cartographic databases, insertions and deletions are rare; databases with census results are static; the same is true for databases that are published on CD-ROMs; databases with spatio/temporal meteorological and environmental data are seldom modified, too. The volume of data in these databases is expected to be enormous, in which case it is crucial to minimize the space overhead of the index.

The *lowx* packed R-tree [25] is the first published packing method for R-trees. However, it suffers from a subtle pitfall. Although it performs very well for point queries on point data, its performance degrades for larger queries. Figures 3 and 4 highlight the problem. Figure 4 shows the leaf nodes of the R-tree that the *lowx* packing method will create for the points of figure 3.

Intuitively, the ordering algorithm should ideally assign nearby points to nearby orders; ignoring the y -coordinate, the *lowx* packed R-tree tends to violate this empirical rule.

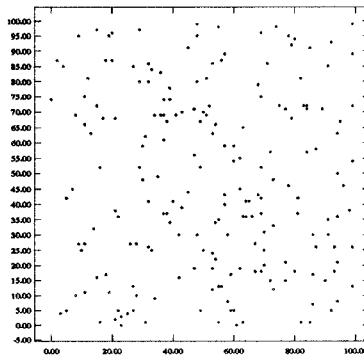


Figure 3: 200 points uniformly distributed

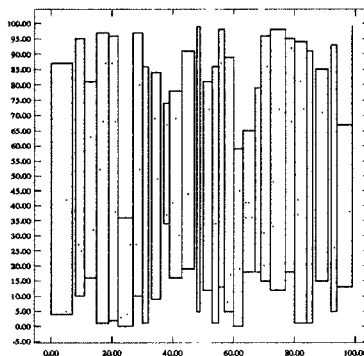


Figure 4: MBR of nodes generated by the 'lowx packed R-tree' algorithm

In order to cluster the data in a better way than the *lowx* packed R-trees, we propose to use space filling curves (or fractals), and specifically, the Hilbert curve. A space filling curve visits all the points in a k -dimensional grid exactly once and never crosses itself. The Z-order (or Morton key order, or bit-interleaving, or Peano curve), the Hilbert curve, and the Gray-code curve [6] are examples of space filling curves. In [7], it was shown experimentally that the Hilbert curve achieves the best clustering among the three above methods.

Next we provide a brief introduction to the Hilbert curve: The basic Hilbert curve on a 2×2 grid, denoted by H_1 , is shown in Figure 5. To derive a curve of order i , each vertex of the basic curve is replaced by the curve of order $i - 1$, which may be appropriately rotated and/or reflected. Figure 5 also shows the Hilbert curves of order 2 and 3. When the order of the curve tends to infinity, the resulting curve is a *fractal*, with a fractal dimension of 2 [21]. The Hilbert curve can be

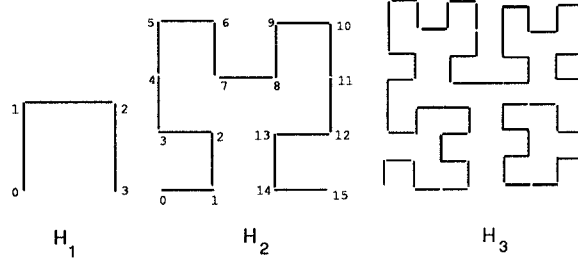


Figure 5: Hilbert Curves of order 1, 2 and 3

generalized for higher dimensionalities. Algorithms to draw the two-dimensional curve of a given order, can be found in [11], [17]. An algorithm for higher dimensionalities is in [5].

The path of a space filling curve imposes a linear ordering on the grid points, which may be calculated by starting at one end of the curve and following the path to the other end. Figure 5 shows one such ordering for a 4×4 grid (see curve H_2). For example the point (0,0) on the H_2 curve has a hilbert value of 0, while the point (1,1) has a hilbert value of 2.

After this preliminary material, we are in a position now to describe the proposed methods. Exploiting the superior clustering that the Hilbert curve can achieve, we impose a linear ordering on the data rectangles and then we traverse the sorted list, assigning each set of C rectangles to a node in the R-tree. The final result is that the set of data rectangles on the same node will be close to each other in the linear ordering, and most likely in the native space; thus the resulting R-tree nodes will have smaller areas. Figure ?? illustrates the intuitive reasons that our hilbert-based methods will result in good performance. The data are points (the same points with Figure 3 and Figure 4). We see that, by grouping the points according to their hilbert values, the MBRs of the resulting R-tree leaf nodes tend to be small *square*-like rectangles. This indicates that the nodes will likely have small area and small perimeter, both of which properties will lead to better R-tree nodes.

We studied several methods to sort the data rectangles. All of them use the same algorithm (see Figure 7) to build the R-tree. The only point that the proposed hilbert-based methods distinguish themselves from each other is on the way they compute the hilbert value of a rectangle. We examine the following alternatives:

4d Hilbert through corners ("4D-xy"): Each data rectangle is mapped to a point in four dimensional space formed by the lower left corner and the upper right corner namely (lowx, lowy, highx, highy). The hilbert value of this 4-dimensional point is the hilbert value of the rectangle.

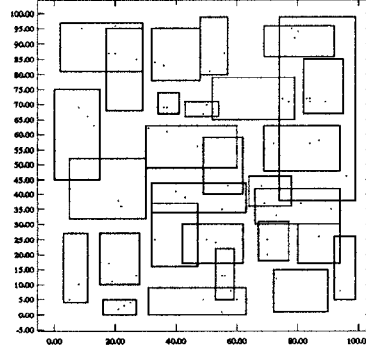


Figure 6: 200 points grouped in *R*-tree nodes, sorted on Hilbert value

4-d Hilbert through center and diameter ("4D-cd"): Each data rectangle is mapped to the 4-dimensional point (c_x, c_y, d_x, d_y) , where c_x, c_y are the coordinates of the center of the rectangle and d_x, d_y the 'diameters' or sides of the rectangle. As in 4D-xy, the hilbert value of this 4-dimensional point is the hilbert value of the rectangle.

2-d Hilbert through Centers Only ("2D-c"): Each data rectangle is represented by its *center only*; the hilbert value of the center is the hilbert value of the rectangle.

For the sake of comparison, we also examined a method that uses the Peano curve, or 'z-ordering', despite the fact that the z-ordering achieves inferior clustering compared to the hilbert curve. The z-value of a point is computed by bit-interleaving the binary representation of its x and y coordinates. For example, in Figure 8, the point (0,0) has a z-value of 0, while the point (1,3) has a z-value of 7.

Z-order through Centers only ("2Dz-c"): The value of the rectangle is the z-value of its center.

Table 1 gives a list of the methods we compared, along with a brief description of each. The new methods are in italics; *R*-tree methods for static environments are above the double horizontal line; the rest can be applied for dynamic environments, as well. The simulation experiments are presented in subsection 5.1.

4 Hilbert R-trees (Dynamic case)

As we shall see in subsection 5.1, the static Hilbert *R*-tree with the '2D-c' ordering method achieves the best results, outperforming the *R**-trees by up to 36%. This hints that a Hilbert *R*-tree

Algorithm Generic-Pack: packs rectangles into an R-tree

Step 1. Calculate the ordering value for each data rectangle

Step 2. Sort data rectangles on ascending ordering values

Step 3. */* Create leaf nodes (level $l=0$) */*

While (there are more rectangles)

 generate a new R-tree node

 assign the next C rectangles to this node

Step 4. */* Create nodes at higher level ($l+1$) */*

While (there are > 1 nodes at level l)

 sort nodes at level $l \geq 0$ on ascending *creation* time

 repeat Step 3

Figure 7: Pseudo-code of the packing algorithm

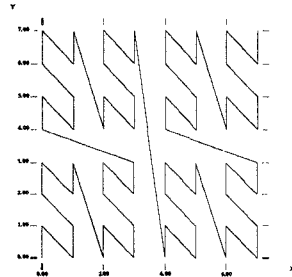


Figure 8: Peano (or z-order) curve of order 3

Method name	Description
<i>2D-c</i>	<i>sorts on the 2d-hilbert value of the centers (c_x, c_y)</i>
<i>4D-xy</i>	<i>sorts on the 4-d hilbert value of the two corners. i.e., $(low_x, low_y, high_x, high_y)$</i>
<i>4D-cd</i>	<i>sorts on 4-d hilbert value of the center and diameters, i.e., (c_x, c_y, d_x, d_y)</i>
<i>2Dz-c</i>	<i>sorts on the z-value of the center (c_x, c_y)</i>
lowx packed R-tree [25]	sorts on the x coordinate of the lower left corner
linear-split R-tree [14]	Guttman's R-tree, with linear split
quadratic-split R-tree [14]	Guttman's R-tree with quadratic split
R^* -tree [3]	R-tree variant, better packing, forced reinsert

Table 1: List of methods - the proposed ones are in *italics*

with lower utilization will still outperform the R^* -trees. In this section we exactly sacrifice some utilization and speed, to make our method dynamic. Next, we discuss the structural modifications and the algorithms for searching, insertion, deletion, and overflow handling.

The main idea is to create a tree structure that can

- behave like an R-tree on search
- behave like a B^* -tree with deferred splitting on insertion, using the '2D-c' value of the inserted data rectangle as the primary key. (Deletion will be handled in a similar way).

These goals can be achieved as follows: for every node n of the R-tree, we store (a) the MBR, as an R-tree would and (b) the *Largest Hilbert Value (LHV)* of the data rectangles that belong to the subtree with root n

Specifically, the Hilbert R-tree has the following structure. Exactly like an R-tree, a leaf node contains at most C_l entries each of the form

$$(R, obj - id)$$

where C_l is the capacity of the leaf, R is the MBR of the real object $(x_{low}, x_{high}, y_{low}, y_{high})$ and $obj - id$ is a pointer to the object description record. The main difference with R- and R^* -trees is that nonleaf nodes also contain information about the LHVs. Thus, a non-leaf node in the Hilbert R-tree contains at most C_n entries of the form

$$(R, ptr, LHV)$$

where C_n is the capacity of a non-leaf node, R is the MBR that encloses all the children of that node, ptr is a pointer to the child node, and LHV is the largest hilbert value among the *data* rectangles enclosed by R . Notice that we *never* calculate or use the hilbert values of the MBRs. Figure 9 illustrates some rectangles, organized in a Hilbert R-tree. The hilbert values of the centers are the numbers by the bullets (shown only for the shaded father node 'II'). The LHV's are in [brackets]. Figure 10 shows how is the tree of Figure 9 stored on the disk; the contents of the shaded father node 'II' are shown in more detail. Every data rectangle in node 'I' has hilbert value ≤ 33 ; everything in node 'II' has hilbert value greater than 33 and ≤ 107 etc.

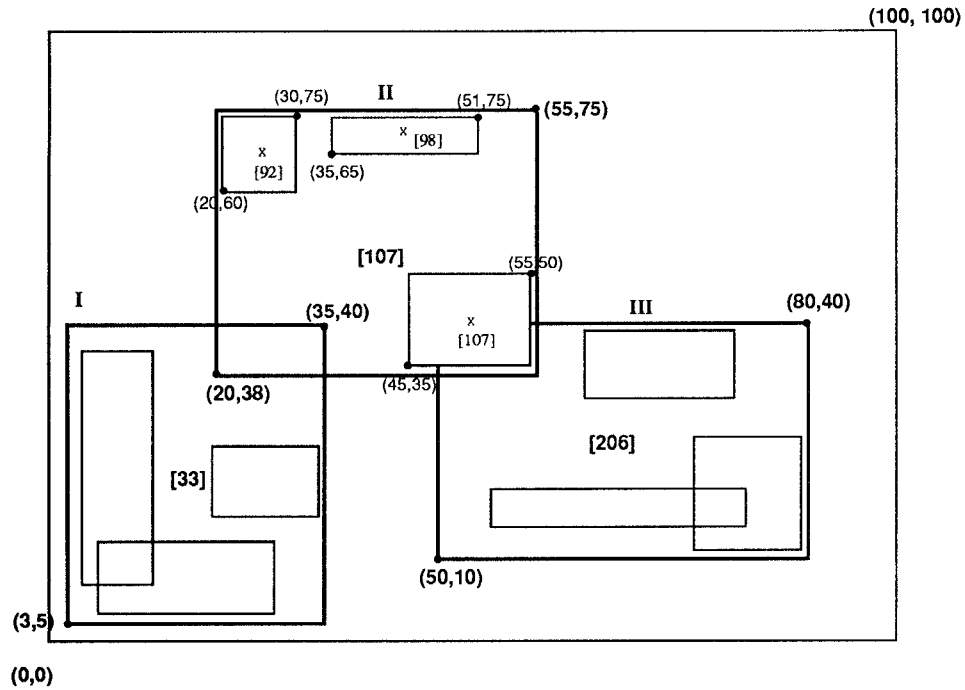


Figure 9: Data rectangles organized in a Hilbert R-tree

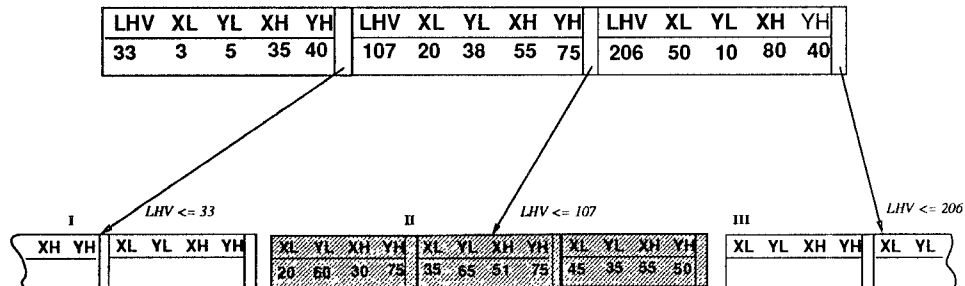


Figure 10: The file structure for the previous Hilbert R-tree

Before we continue, we list some definitions related to the B^* -trees. A plain B -tree splits its nodes on overflow, turning 1 node to 2. We call this policy a *1-to-2* splitting policy. B^* -trees defer the split, typically waiting until they turn 2 nodes into 3. We refer to it as the *2-to-3* splitting policy. In general, we can have an s -to- $(s+1)$ splitting policy; we refer to s as the *order of the splitting policy*. To implement the order- s splitting policy, the overflowing node tries to push some of its entries to one of its $s - 1$ siblings; if all of them are full, then we have an s -to- $(s+1)$ split. We refer to these $s - 1$ siblings as the *cooperating siblings* of a given node.

Given that the search algorithm is identical to the R-tree search algorithm and that the insertion/split/deletion algorithms are identical to the B^* -tree algorithms, we do not present the pseudo-code, for brevity.

5 Experimental results

To assess the merit of our proposed hilbert-based R-trees, we ran simulation experiments on a two-dimensional native space. We compared our methods against the linear-split R-tree, the quadratic-split R-tree, the R^* -tree, and the *lowx* packed R-tree. Since the CPU time required to process the node is negligible, we based our comparison on the number of nodes (=pages) retrieved by range queries.

Without loss of generality, the address space was normalized to the unit square. There are several factors that affect the search time; we studied the following ones:

Data items: points and/or rectangles and/or line segments (represented by their MBR)

File size: ranged from 10,000 - 100,000 records

Query area $Q_{area} = q_x \times q_y$: ranged from 0 - 0.3 of the space size

Page size p: varied in the range 1Kb - 4Kb

Another important factor, which is derived from N and the average area a of the data rectangles, is the ‘data density’ d (or ‘cover quotient’) of the data rectangles. This is the sum of the areas of the data rectangles in the unit square, or equivalently, the average number of rectangles that cover a randomly selected point. Mathematically: $d = N \times a$. For the selected values of N and a , the data density ranges from 0.25 - 2.0.

To compare the performance of our proposed structures we used 5 data files that contained different types of data: points, rectangles, lines, or mixed. Specifically, we used:

A) Real Data: we used real data from the TIGER system of the U.S. Bureau of Census. An important observation is that the data in the TIGER dataset follow a highly *skewed* distribution.

MGCounty : This file consists of 39717 line segments, representing the roads of Montgomery county in Maryland. Using the minimum bounding rectangles of the segments, we obtained 39717 rectangles, with data density $d = 0.35$. We refer to this dataset as the ‘*MGCounty*’ dataset.

LBeach : consists of 53145 line segments, representing the roads of Long Beach, California. The data density of the MBR that covers these line segments is $d = 0.15$. We refer to this dataset as the ‘*LBeach*’ dataset.

B) Synthetic Data: The reason for using synthetic data is that we can control the parameters (data density, number of rectangles, ratio of points to rectangles etc.).

Points : This file contains 75,000 uniformly distributed points.

Rects : This file contains 100,000 rectangles, no points. The centers of the rectangles are uniformly distributed in the unit square. the data density is $d = 1.0$

Mix : This file contains a mix of points and rectangles; specifically 50,000 points and 10,000 rectangles; the data density is $d = 0.029$.

The query rectangles were squares with side q_s , their centers are uniformly distributed in the unit square. For each experiment, 50 randomly generated queries were asked and the results were averaged. In our plots we omit the results of the linear-split R-tree, because the quadratic-split R-tree consistently outperformed it. The exponential-split R-tree was very slow in building the tree, and it was not used.

In the following subsections we present two groups of experiments to evaluate our method. in a static and in a dynamic environment, respectively.

5.1 Static Case

In this section we evaluate the performance of our Hilbert R-tree for static environment. We carried experiments to (a) compare the response time of the best of our methods (2D-c) with the response time of older R-tree variants (dynamic or static) (b) compare our hilbert-based packing schemes against each other, to pinpoint the best.

5.1.1 Comparison of 2D-c hilbert packed R-tree vs. older R-tree variants

Here we illustrate that the 2D-c packing method gives better response times than older R-tree variants, namely, the *lowx* packed R-tree, the quadratic-split R-tree and the R^* -tree.

To avoid cluttering the plots, we only plot the best of our proposed algorithms, namely the one using the ‘2D-c’ heuristic. The detailed results for the other, hilbert-based packing algorithms are presented in the next subsection.

We performed experiment with all the sets of data. The results were similar. For brevity, we only show the results for the ‘MGCounty’ dataset, with line segments from a TIGER file. The rest of the graphs are in a technical report [8]. For the MGCounty dataset, Figure 11 plots the number of pages retrieved by a range query as a function of the area of the query. Notice that for point queries, all methods performs almost the same, with small differences. However, for slightly larger queries, the proposed 2D-c hilbert packed R-tree is the clear winner. The performance gap increases with the area of the queries. The proposed method achieves up to 36% improvement over the next best method (R^* -tree) and up to 58% improvement over the *lowx* packed R-tree.

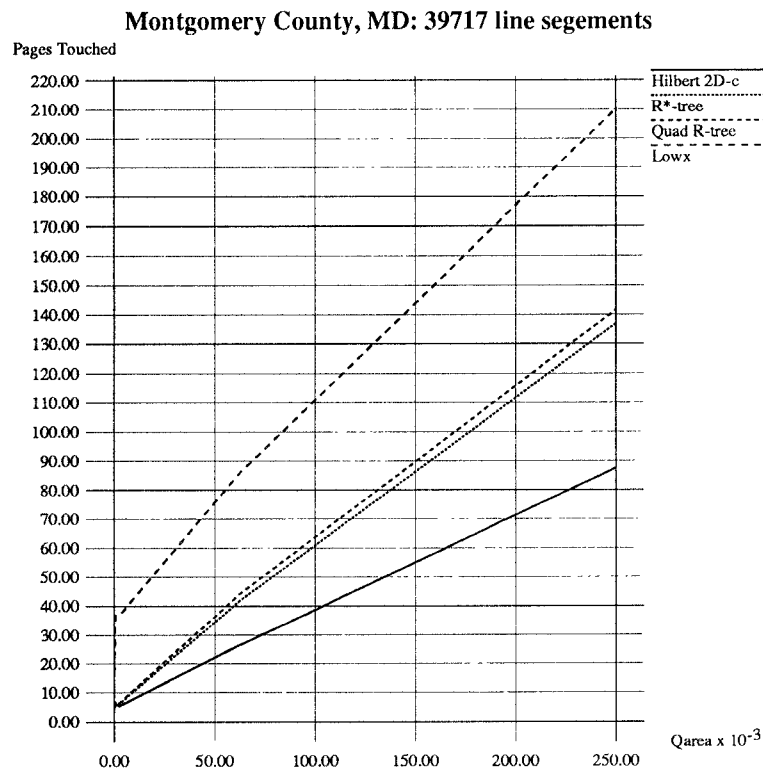


Figure 11: *STATIC ENVIRONMENT* - real data (MGCounty dataset)

5.1.2 Comparison of hilbert-based packing schemes

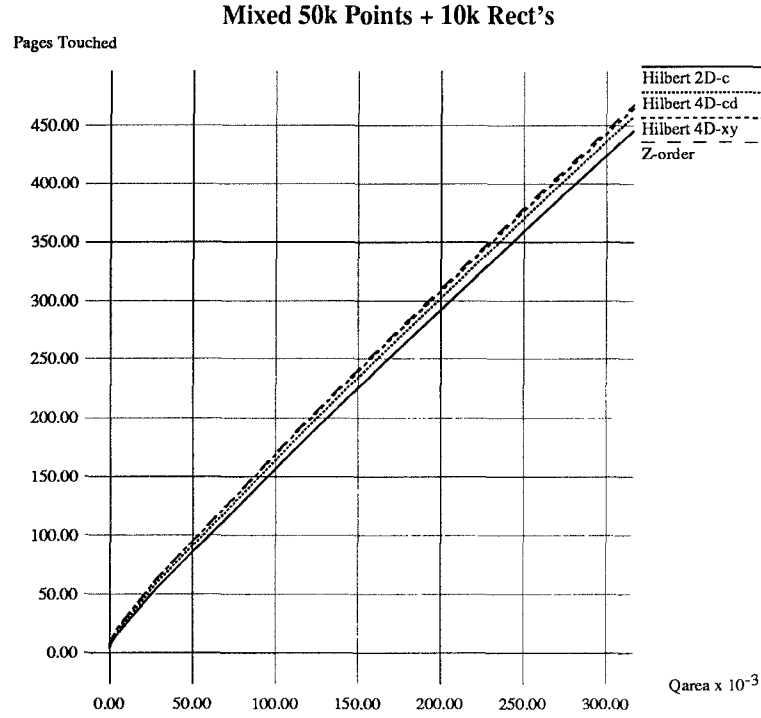


Figure 12: *STATIC ENVIRONMENT - Comparison Between Schemes that use Space Filling Curves*

Here we compare all the packing heuristics that we have introduced in this paper, namely 2D-c , 4D-xy, 4D-cd and the only heuristic that uses the z-ordering, 2Dz-c. Table 1 contains a list of these methods, along with a brief description. Figure 12 plots the response time versus the query area for all of these heuristics.

The (synthetic) data file consists of 50K points and 10K rectangles. The page size $p = 1\text{Kb}$. The differences between the alternative methods are small. However, from Figure 12 we see that (2D-c) does better, especially for large queries. The next best method is the (4D-cd), which uses a 4-d hilbert curve on the parameter space (center-x, center-y, diameter-x, diameter-y). The remaining methods, 2Dz-c and 4D-xy, almost tie in the third place.

The relative ranking of the methods was the same for every dataset we tried; we omit the plots for brevity.

5.2 Dynamic Case

In this section we evaluate our proposed index 'Hilbert R-tree' for dynamic environment. We carried out many experiments to compare the performance of the Hilbert R-tree against the other R-tree

variants. Before doing a full-fledged implementation of the s -to- $(s + 1)$ split policy, we did some experiments to assess the search performance of the resulting Hilbert R-tree. In our experiments, we sorted the data on their hilbert value, and loaded them into an R-tree, with node utilization $u\%$. In order to calculate the average utilization for the splitting policy of order s , we used a result by Leung [19]. It states that the average space utilization for $B^* - tree$ is given by the following equation :

$$u = f \times \frac{\ln(1/f)}{1 - f} \quad (1)$$

where f is the minimum fullness $0 < f < 1$. For a splitting policy of order s , we have

$$f = \frac{s}{s + 1} \quad (2)$$

Since a real Hilbert R-tree will not have constant utilization across its nodes, we also loaded the (hilbert-sorted) data in an R-tree, where the *average* node utilization was u , while the individual node utilization varied uniformly in the range $(s/(s + 1) - 1)$. The variability of the utilization made little difference in the response performance, in favor of the 'constant' utilization across nodes. Nevertheless, in our experiments we used the least favorable case, ie., with utilization varying across nodes. Unless otherwise specified, in the upcoming figures we used a '3-to-4' splitting policy, ($s=3$), with 86% utilization.

We compared our Hilbert R-tree against the original R-tree (both linear and quadratic split) and the $R^* - tree$. In the following subsection we present experiments to (a) compare our proposed Dynamic Hilbert R-tree against other R-tree variants and (b) show the effect of the different split policies on the performance of the proposed method.

5.2.1 Comparison of the Dynamic Hilbert R-tree vs. other R-tree variants

In this section we show the performance superiority of our Hilbert R-tree over the $R^* - tree$ and the quadratic R-tree, which are the most successful variants of the R-tree. We present experiments with all five datasets, namely: MGCounty, LBeach, Mix, Rects, and Points. The results are shown in Figures 13 - 17 respectively.

In all the experiment the Hilbert R-tree is the clear winner. It achieves up to 28% improvement in the response time over $R^* - tree$ (the best known R-tree variant). This maximum gain is achieved for the 'MGCounty' dataset (Figure 13). It is interesting to notice that the performance gap is larger for the real data, whose main difference from the synthetic one is that it is skewed, as opposed to uniform. Thus, we can conjecture that the skewness of the data favors the Hilbert R-tree.

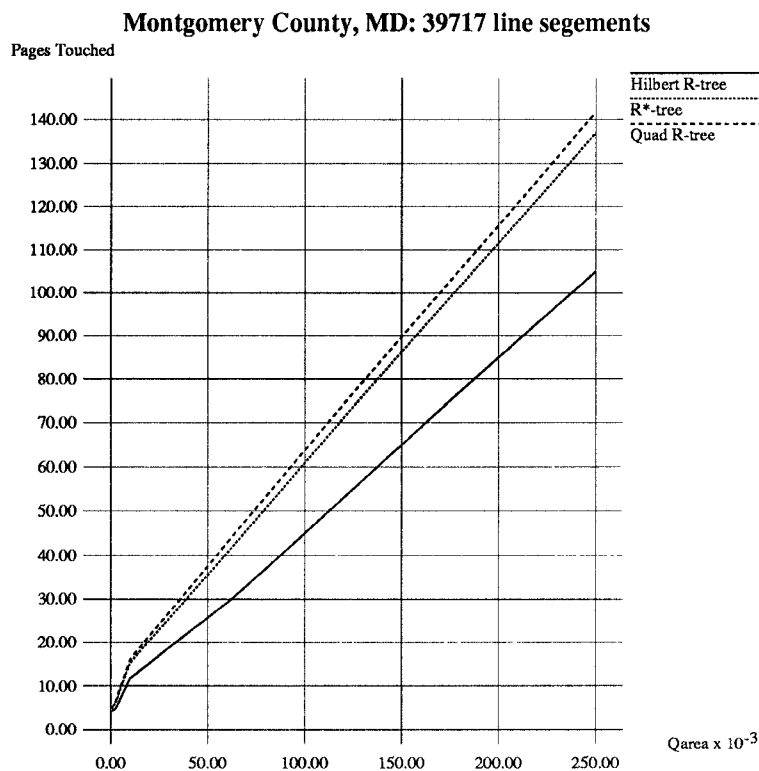


Figure 13: *DYNAMIC ENVIRONMENT - Montgomery County, MD*

5.2.2 The effect of the split policy on the performance

In this section we examine the different split policies for the Dynamic Hilbert R-tree and its effects on the response time. As before, we used 1 to estimate the average utilization. Figure 18 shows the response time as a function of the query size for 1-2, 2-3, 3-4 and 4-5 split policies which results in (according to equation 1 the following average node utilization of 69%, 81%, 86%, and 89% respectively. We also, show the case where we have maximum space utilization 100%. As expected, the response time for the range queries improves with the average node utilization. However, there seems to be a point of diminishing returns as s increases. For this reason, we recommend the '3-to-4' splitting policy ($s=3$), which strikes a balance between insertion speed (which decreases with s) and performance, which increases with s .

6 Conclusions

The major idea in this paper is to introduce a 'good' ordering among rectangles. By simply defining an ordering, the R-tree structure is amenable to deferred splitting, which can make the utilization

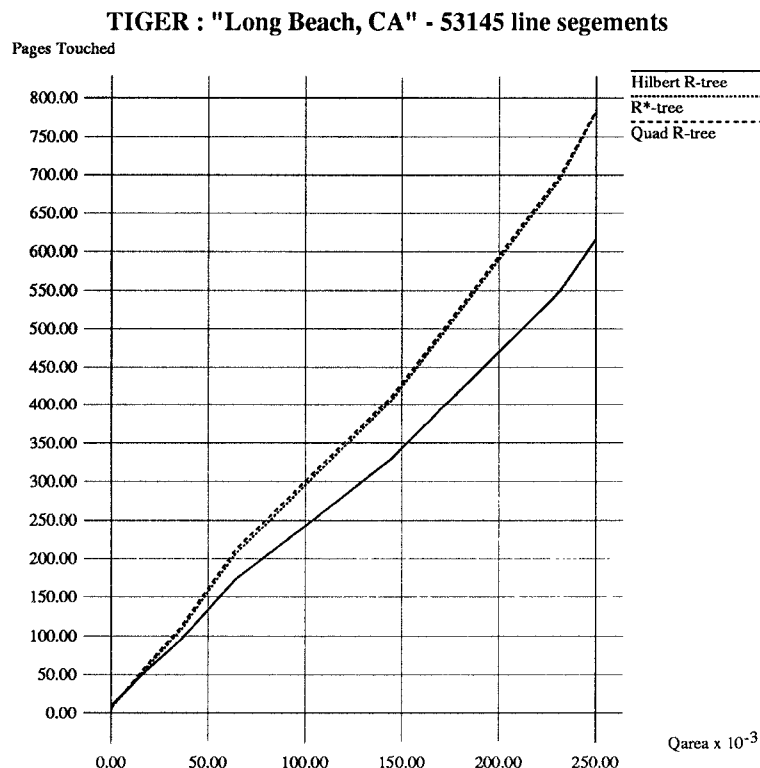


Figure 14: *DYNAMIC ENVIRONMENT - Long Beach, CA*

approach the 100% mark as closely as we want. Better packing results in a shallower tree and a higher fanout. If the ordering happens to be 'good', that is, to group similar rectangles together, then the R-tree will in addition have nodes with small MBRs, and eventually, fast response times.

We experimented with several orderings, and we discovered that the '2D-c' ordering (that is, the one using the hilbert value of the center of the rectangle) gives the best results. In the static case, it outperforms all the known R-tree methods, either dynamic or static, with up to 36% savings over the second best competitor (the R^* -tree).

We also designed the Hilbert R-tree, a dynamic tree structure that behaves like an R-tree on searches and as a B^* -tree on insertions/deletions/splits. Experiments on real and synthetic data showed that the proposed Hilbert R-tree with the '3-to-4' splitting policy consistently outperforms all the dynamic R-tree methods, with up to 28% savings.

Future work could include the analysis of Hilbert R-trees, possibly extending the analysis on B^* -trees.

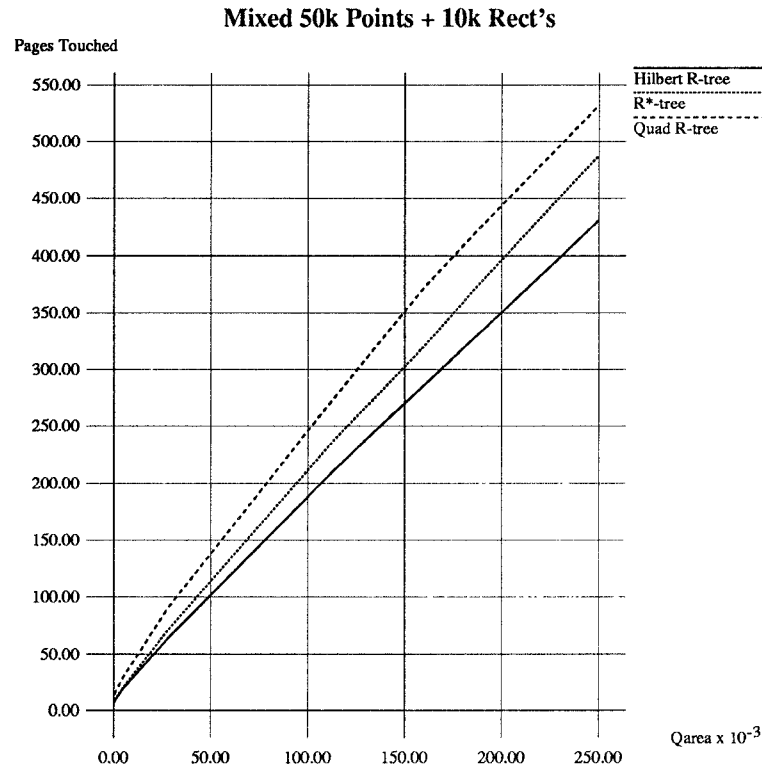


Figure 15: *DYNAMIC ENVIRONMENT - Points and Rectangles*

References

- [1] Walid G. Aref and Hanan Samet. Optimization strategies for spatial query processing. *Proc. of VLDB (Very Large Data Bases)*, pages 81–90, September 1991.
- [2] D. Ballard and C. Brown. *Computer Vision*. Prentice Hall, 1982.
- [3] N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger. The r*-tree: an efficient and robust access method for points and rectangles. *ACM SIGMOD*, pages 322–331, May 1990.
- [4] J.L. Bentley. Multidimensional binary search trees used for associative searching. *CACM*, 18(9):509–517, September 1975.
- [5] T. Bially. Space-filling curves: Their generation and their application to bandwidth reduction. *IEEE Trans. on Information Theory*, IT-15(6):658–664, November 1969.
- [6] C. Faloutsos. Gray codes for partial match and range queries. *IEEE Trans. on Software Engineering*, 14(10):1381–1393, October 1988. early version available as UMIACS-TR-87-4. also CS-TR-1796.

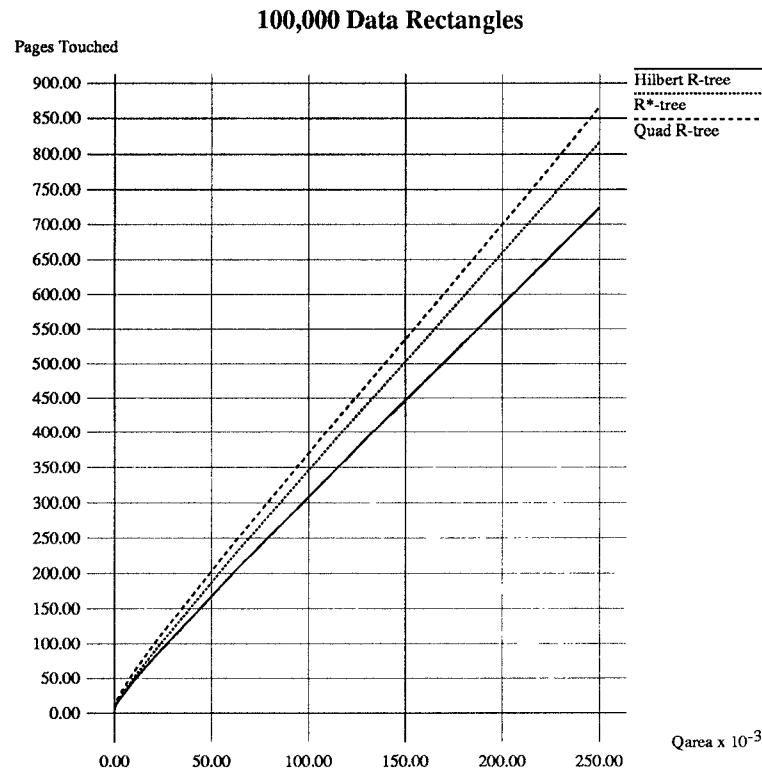


Figure 16: *DYNAMIC ENVIRONMENT - Rectangles Only*

- [7] C. Faloutsos and S. Roseman. Fractals for secondary key retrieval. *Eighth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS)*, pages 247–252, March 1989. also available as UMIACS-TR-89-47 and CS-TR-2242.
- [8] Christos Faloutsos and Ibrahim Kamel. Packed r-trees using fractals. Technical Report TR 93-1, System Research Center (SRC), Univ. of Maryland, College Park, 1993.
- [9] I. Gargantini. An effective way to represent quadrees. *Comm. of ACM (CACM)*, 25(12):905–910, December 1982.
- [10] D. Greene. An implementation and performance analysis of spatial data access methods. *Proc. of Data Engineering*, pages 606–615, 1989.
- [11] J.G. Griffiths. An algorithm for displaying a class of space-filling curves. *Software-Practice and Experience*, 16(5):403–411, May 1986.
- [12] O. Gunther. The cell tree: an index for geometric data. Memorandum No. UCB/ERL M86/89. Univ. of California, Berkeley, December 1986.

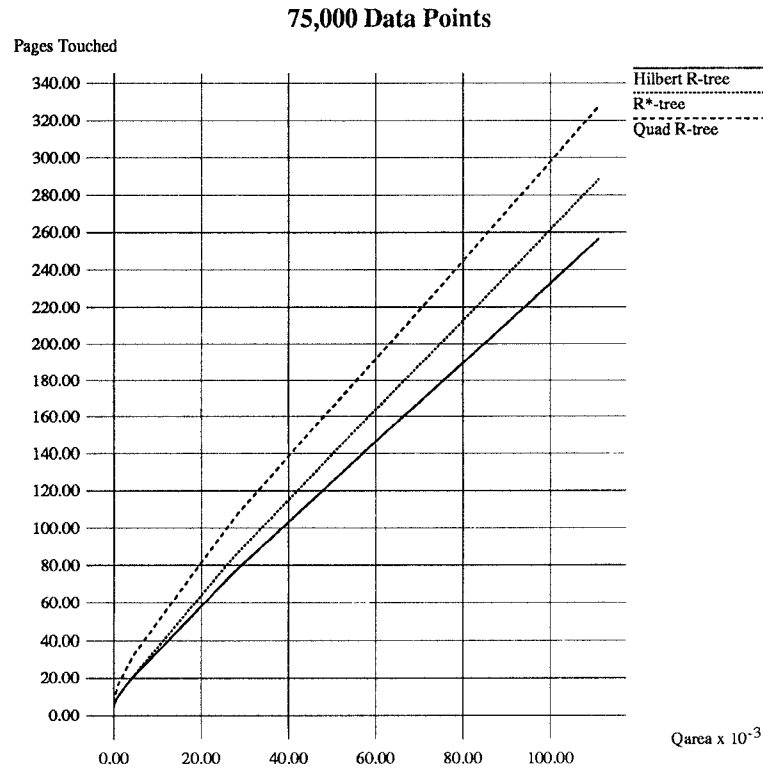


Figure 17: *DYNAMIC ENVIRONMENT - Points Only*

- [13] A. Guttman. *New Features for Relational Database Systems to Support CAD Applications*. PhD thesis, University of California, Berkeley, June 1984.
- [14] A. Guttman. R-trees: a dynamic index structure for spatial searching. *Proc. ACM SIGMOD*, pages 47–57, June 1984.
- [15] K. Hinrichs and J. Nievergelt. The grid file: a data structure to support proximity queries on spatial objects. *Proc. of the WG'83 (Intern. Workshop on Graph Theoretic Concepts in Computer Science)*, pages 100–113, 1983.
- [16] H. V. Jagadish. Spatial search with polyhedra. *Proc. Sixth IEEE Int'l Conf. on Data Engineering*, February 1990.
- [17] H.V. Jagadish. Linear clustering of objects with multiple attributes. *ACM SIGMOD Conf.*, pages 332–342, May 1990.
- [18] Curtis P. Kolovson and Michael Stonebraker. Segment indexes: Dynamic indexing techniques for multi-dimensional interval data. *Proc. ACM SIGMOD*, pages 138–147, May 1991.

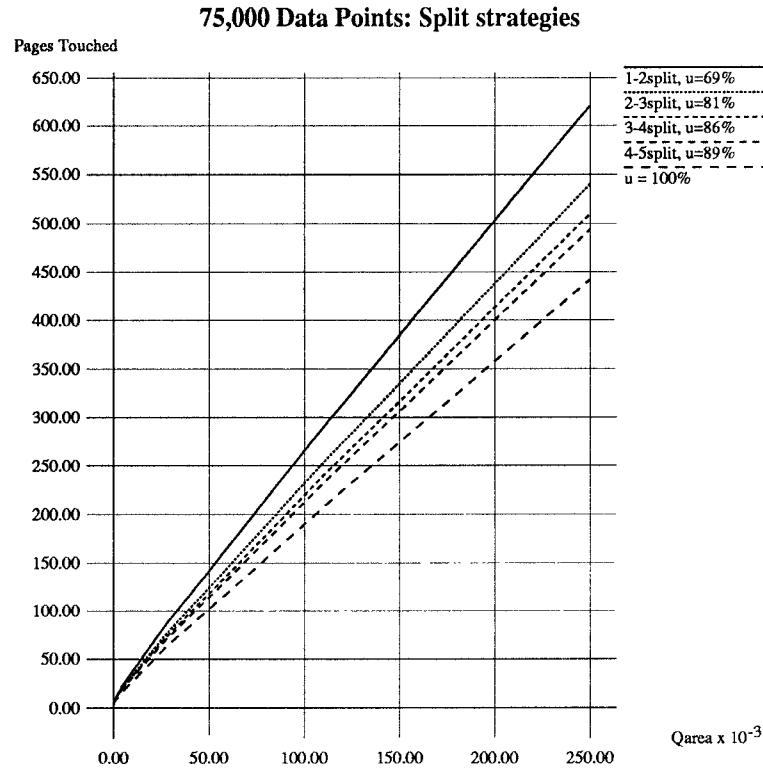


Figure 18: *DYNAMIC ENVIRONMENT* - The effect of the split policy

- [19] Clement Leung. Approximate storage utilization of b-trees: A simple derivation and generalizations. *Information Processing Letters*, 19(4):199–201, November 1984.
- [20] David B. Lomet and Betty Salzberg. The hb-tree: a multiattribute indexing method with good guaranteed performance. *ACM TODS*, 15(4):625–658, December 1990.
- [21] B. Mandelbrot. *Fractal Geometry of Nature*. W.H. Freeman, New York, 1977.
- [22] J. Orenstein. Spatial query processing in an object-oriented database system. *Proc. ACM SIGMOD*, pages 326–336, May 1986.
- [23] J. K. Ousterhout, G. T. Hamachi, R. N. Mayo, W. S. Scott, and G. S. Taylor. Magic: a vlsi layout system. In *21st Design Automation Conference*, pages 152 – 159, Albuquerque, NM, June 1984.
- [24] J.T. Robinson. The k-d-b-tree: a search structure for large multidimensional dynamic indexes. *Proc. ACM SIGMOD*, pages 10–18, 1981.

- [25] N. Roussopoulos and D. Leifker. Direct spatial search on pictorial databases using packed r-trees. *Proc. ACM SIGMOD*, May 1985.
- [26] H. Samet. *The Design and Analysis of Spatial Data Structures*. Addison-Wesley, 1989.
- [27] T. Sellis, N. Roussopoulos, and C. Faloutsos. The r+ tree: a dynamic index for multi-dimensional objects. In *Proc. 13th International Conference on VLDB*, pages 507–518, England,, September 1987. also available as SRC-TR-87-32, UMIACS-TR-87-3, CS-TR-1795.
- [28] M. White. *N-Trees: Large Ordered Indexes for Multi-Dimensional Space*. Application Mathematics Research Staff, Statistical Research Division, U.S. Bureau of the Census, December 1981.

